



ESCUELA DE INGENIERÍA DE
FUENLABRADA

GRADO EN
INGENIERÍA DE ROBÓTICA SOFTWARE

TRABAJO FIN DE GRADO

IMPLEMENTACIÓN DE UN ROBOT POR CABLES PARA EL
CONTROL DE UN EFECTOR FINAL EN DIVERSAS TAREAS

Autor: Alberto León Luengo

Tutor: Dr. Juan Sebastián Cely Gutiérrez

Curso académico 2025-2026



Este trabajo se distribuye bajo los términos de la licencia internacional CC BY-SA International License (Creative Commons Attribution-ShareAlike 4.0). Usted es libre de *(a) compartir*: copiar y redistribuir el material en cualquier medio o formato para cualquier propósito, incluso comercialmente; y *(b) adaptar*: remezclar, transformar y construir a partir del material para cualquier propósito, incluso comercialmente. La licenciente no puede revocar estas libertades en tanto usted siga los términos de la licencia:

- *Atribución.* Usted debe dar crédito de manera adecuada, brindar un enlace a la licencia, e indicar si se han realizado cambios. Puede hacerlo en cualquier forma razonable, pero no de forma tal que sugiera que usted o su uso tienen el apoyo de la licenciente.
- *Compartir igual.* Si remezcla, transforma o crea a partir del material, debe distribuir su contribución bajo la misma licencia del original.

No hay restricciones adicionales — No puede aplicar términos legales ni medidas tecnológicas que restrinjan legalmente a otras a hacer cualquier uso permitido por la licencia.

Agradecimientos

Me gustaría mostrarles todo mi agradecimiento a mis padres, Natividad y Alberto, que han estado siempre apoyándome en todo momento y ayudándome a superar los obstáculos más difíciles a los que he tenido que hacer frente a lo largo de estos seis años de etapa universitaria.

A mi hermana Ainhoa, por ser capaz de sacarme una sonrisa siempre que la situación no ha sido favorable y por ayudarme a salir de mi rutina habitual manteniéndome dentro de mi zona de confort.

A Juan, mi tutor, por el buen trato y todos los nuevos conocimientos y enseñanzas que he podido aprender gracias a él en todo el tiempo que he compartido con él realizando este TFG.

Y por último, a todos los compañeros que me han estado acompañando en la recta final de este viaje, y que gracias al apoyo que ellos me han ofrecido y que yo de la misma forma les he ofrecido a ellos, todos mis objetivos y éxitos esperados han acabado dando sus frutos.

Acrónimos

TFG *Trabajo de Fin de Grado*

CDPR *Cable Driven Parallel Robot*

ROS *Robot Operating System*

RViZ *ROS Visualization*

URDF *Unified Robot Description Format*

Índice general

1. Introducción	1
1.1. Contexto y motivación	1
1.2. Planteamiento del problema	2
1.3. Objetivos	3
1.4. Alcance y limitaciones	3
1.5. Metodología	4
1.6. Descripción inicial del sistema	4
2. Fundamentos teóricos	5
2.1. Introducción	5
2.2. Robots paralelos accionados por cables (CDPR)	5
2.3. Configuraciones típicas	5
2.4. Modelo cinemático	6
2.5. Control y planificación de trayectorias	6
2.6. Frameworks robóticos	6
2.7. Aplicaciones y líneas de investigación actuales	7
2.8. Relación con el trabajo de fin de grado desarrollado	7
3. Plataforma de desarrollo	8
3.1. Introducción	8
3.2. Funcionamiento de un CDPR bidimensional	8
3.3. Arquitectura	8
3.4. Componentes software	10
3.5. Sistema hardware	10
3.6. Flujo general de funcionamiento	11
3.7. Relación entre módulos	12
4. Modelo cinemático	13
4.1. Introducción	13

4.2. Descripción geométrica del sistema	13
4.3. Definición de los puntos de anclaje	14
4.4. Cinemática inversa	14
4.5. Cálculo de los ángulos de los cables	15
4.6. Relación cables-poleas	15
4.7. Consideraciones y limitaciones	16
5. Simulación y validación del modelo	17
5.1. Introducción	17
5.2. Simulación inicial en MATLAB	17
5.3. Implementación del modelo en Python	18
5.4. Implementación del modelo en C++	19
5.5. Comparación y validación de resultados	20
5.6. Análisis de errores y limitaciones	20
6. Integración del sistema en ROS 2	21
6.1. Introducción	21
6.2. Estructura del paquete ROS 2	21
6.3. Nodo de control del CDPR	22
6.4. Comunicación mediante topics	24
6.5. Gestión de trayectorias y movimientos	25
6.6. Visualización del sistema en RViZ	26
6.7. Compilación y ejecución del sistema	26
7. Desarrollo hardware	27
7.1. Introducción	27
7.2. Diseño mecánico del sistema	27
7.3. Sistema electrónico	27
7.4. Programación del microcontrolador	27
7.5. Integración hardware-software	27
7.6. Consideraciones prácticas y limitaciones del prototipo	27
7.7. Resumen	27
8. Pruebas experimentales y validación	28
8.1. Introducción	28
8.2. Metodología de pruebas	28
8.3. Pruebas de posicionamiento del efector final	28

8.4. Prueba de trayectorias	28
8.5. Comparación entre simulación y resultados	28
8.6. Análisis de errores y fuentes de incertidumbre	28
8.7. Resumen	28
9. Discusión de resultados	29
9.1. Introducción	29
9.2. Evaluación del cumplimiento de los objetivos	29
9.3. Análisis de los resultados de la simulación	29
9.4. Análisis de los resultados experimentales	29
9.5. Comparación entre comportamiento teórico y real	29
9.6. Limitaciones del sistema desarrollado	29
9.7. Propuesta de mejora y líneas de evolución	29
9.8. Resumen	30
10. Conclusiones y mejoras a futuro	31
10.1. Introducción	31
10.2. Conclusiones generales	31
10.3. Grado de cumplimiento de los objetivos	31
10.4. Aportaciones del trabajo	31
10.5. Limitaciones finales del sistema	31
10.6. Mejoras a futuro	31
10.7. Cierre del trabajo	31
11. Uso e impacto de la IA y los LLM en el TFG	32
11.1. Introducción	32
11.2. Herramientas utilizadas	32
11.3. Ámbitos de utilización	33
11.4. Nivel de dependencia	34
11.5. Impacto en el desarrollo del TFG	34
11.6. Limitaciones y consideraciones éticas	35
11.7. Conclusión	35
12. Conceptos comunes abordados en grado y TFG	36
12.1. Introducción	36
12.2. Relación con asignaturas de primer curso	36
12.3. Relación con asignaturas de segundo curso	37

12.4. Relación con asignaturas de tercer curso	37
12.5. Relación con asignaturas de cuarto curso	38
12.6. Integración global de conocimientos en el TFG	38
12.7. Conclusión	39
Bibliografía	40

Índice de figuras

1.1. Ejemplo visual de un CDPR de dos grados de libertad	2
3.1. Arquitectura general del sistema en ROS 2	9
3.2. Flujo general de funcionamiento del sistema	11
5.1. Simulación inicial en MATLAB	18
5.2. Implementación y simulación en Python	19
5.3. Implementación y simulación en C++	20
6.1. Estructura del nodo cdpr_controller	23
6.2. Visualización en RViZ durante la ejecución de una trayectoria	24
6.3. Comunicación mediante topics en ROS 2	24

Listado de ecuaciones

4.1. Vector de posición del efector final	13
4.2. Puntos de anclaje entre cada cable y el efector final	14
4.3. Longitudes de cada cable	14
4.4. Orientación angular de cada cable	15
4.5. Relación entre desplazamiento y rotación de cada cable y polea	15

Capítulo 1

Introducción

1.1. Contexto y motivación

Los robots paralelos accionados por cables, también conocidos como Cable Driven Parallel Robots (CDPR), han despertado bastante interés en los últimos años tanto en el ámbito académico como en el ámbito industrial. Este tipo de robots basa su funcionamiento en el uso de cables que pueden ser recogidos o elongados por poleas con el objetivo de desplazar el efector final que une todos los cables a una posición específica, obteniendo como resultado una estructura ligera, escalable, y lo más importante de todo, capaz de realizar grandes volúmenes de trabajo en comparación con otros tipos de robots.

Durante los últimos seis años, los cuales he dedicado enteramente a mi formación en el Grado en Ingeniería de Robótica Software, y tras haber abordado diferentes campos de la robótica, he decidido decantarme por este tipo de robots como tema principal de mi TFG, ya que combina conceptos de modelado matemático y cinemático, el uso de un lenguaje de programación de alto nivel como lo es Python, y una de las arquitecturas software por excelencia utilizadas en el mundo de la robótica: ROS 2. Este Trabajo de Fin de Grado tiene como objetivo principal la profundización en el diseño y el desarrollo de un CDPR de dos grados de libertad desde un punto de vista práctico y orientado al software.

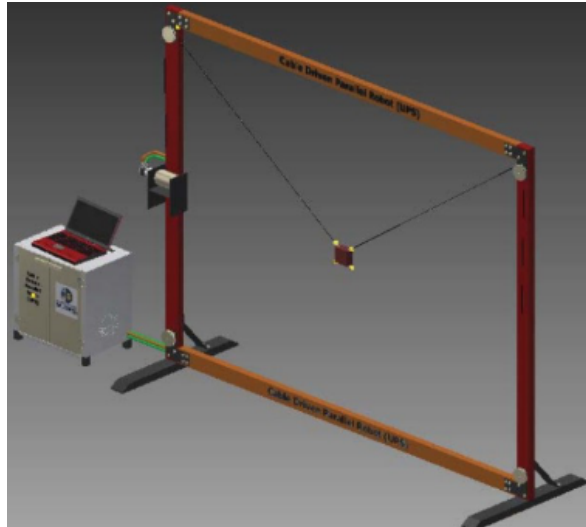


Figura 1.1: Ejemplo visual de un CDPR de dos grados de libertad

Y por último, es importante mencionar que la principal motivación que me ha llevado a elegir este tipo de robots como tema principal de mi TFG es que el comportamiento global del sistema viene determinado principalmente por la correcta formulación del modelo cinemático, además de todo lo que respecta a la implementación software.

1.2. Planteamiento del problema

El problema principal que se aborda en este TFG es el control de la posición de un efector final suspendido mediante dos cables ubicados en una estructura fija, en este caso una pizarra móvil con ruedas, a través de poleas, donde el sistema debe ser capaz de desplazar el efector final hacia una o varias posiciones específicas, siendo capaz de definir trayectorias en caso de que el efector final pase por más de una posición, todo ello dentro de una región bidimensional cuadrada cuya área cuadrada viene definida por la distancia que separa ambas poleas.

Sabiendo esto, es necesario establecer una relación entre la región por la que se mueve el efector final y las variables físicas del sistema, como por ejemplo las longitudes de los cables y el ángulo que debe rotar cada polea para desplazar el efector final a una posición específica. Esta relación debe ser lo suficientemente robusta de forma que pueda implementarse en un entorno real, y lo suficientemente flexible como para poder integrarla en software robótico.

En este TFG me he centrado principalmente en desarrollar todo el software necesario para resolver este problema, dejando la implementación del sistema real como una fase posterior, actualmente en proceso de definición y que abordaré en conjunto con mi tutor de TFG, Juan Sebastián Cely Gutiérrez.

1.3. Objetivos

El objetivo principal de este TFG consiste en el diseño, implementación y validación de un sistema capaz de controlar un CDPR bidimensional, comenzando por la definición del modelo cinemático, pasando por su integración en ROS 2 y finalizando con la construcción de un prototipo del sistema en un entorno real utilizando elementos hardware.

En términos específicos, los objetivos a destacar son los siguientes:

- Desarrollo de un modelo cinemático que relacione la posición en la que se encuentre el efector final con las longitudes de los cables y los ángulos de giro de las poleas en cada momento.
- Validación del modelo desarrollado en distintos lenguajes de programación.
- Implementación del modelo cinemático en un nodo ROS 2 que ofrezca información sobre el movimiento y las trayectorias realizadas en cada momento.
- Integración completa del sistema en ROS 2 utilizando RViZ como herramienta de visualización.
- Integración del software desarrollado en un sistema hardware real.

1.4. Alcance y limitaciones

El alcance de este TFG se centra principalmente en la parte software del sistema, dentro de la cual se ha desarrollado y validado un modelo cinemático, además de su integración en ROS 2. Sin embargo, no se ha profundizado en otros aspectos como el modelado de fuerzas, las tensiones de los cables, la estructuración del sistema de forma que este sea de tipo "lazo cerrado", o una interacción más detallada con los sensores en el sistema real, ya que en este caso, éste está compuesto por dos servomotores como únicos elementos hardware.

Por tanto, aunque se presente una descripción del sistema hardware, tanto la construcción del robot real como su puesta en funcionamiento quedan algo más apartadas. Todas estas limitaciones han sido asumidas desde el principio, ya que otro de los objetivos principales de este TFG comprende que toda la parte del desarrollo software sea sólida.

1.5. Metodología

La metodología de este TFG ha sido iterativa e incremental en función de las fases que se han ido abordando. En primer lugar, se ha llevado a cabo la formulación del modelo cinemático del sistema para su posterior validación en pruebas realizadas en diferentes lenguajes de programación como han sido MATLAB, Python y C++ respectivamente, lo que ha permitido detectar y corregir pequeños errores de planteamiento de forma que no se acarreasen hasta fases más avanzadas. Una vez validado el modelo cinemático, se ha procedido a su integración en ROS 2, donde se ha desarrollado un nodo capaz de generar trayectorias y visualizarlas a través de RViZ, lo que ha permitido mantener una separación clara entre validación, simulación e integración.

1.6. Descripción inicial del sistema

Aunque la parte hardware del sistema se encuentre en una fase preliminar, se ha planteado un diseño consistente en consonancia con el modelo software desarrollado, en el que el sistema está compuesto principalmente por una estructura rígida, en este caso una pizarra móvil con ruedas, que soporta dos poleas fijas ubicadas en las esquinas superiores de la misma, a las cuales se conectan los cables encargados de desplazar el efector final.

El movimiento de las poleas se lleva a cabo utilizando servomotores, cuyo tipo y características serán determinados en fases posteriores de este TFG, así como el diseño del efector final y el resto de materiales empleados, los cuales se detallarán una vez finalizada la fase de construcción del sistema real.

Capítulo 2

Fundamentos teóricos

2.1. Introducción

Los robots paralelos accionados por cables (CDPR) han experimentado un gran crecimiento en los últimos años. Comprenden conceptos de robótica paralela, modelado geométrico y mecánica ligera, convirtiéndolos en un tipo de robots muy atractivo tanto para proyectos de investigación como para aplicaciones prácticas.

Este capítulo presenta el contexto teórico que comprende este TFG, además de abordar conceptos fundamentales sobre los CDPR, sus configuraciones más habituales y sus principales líneas de investigación, todo ello relacionado con el enfoque software adaptado.

2.2. Robots paralelos accionados por cables (CDPR)

Un CDPR se caracteriza por utilizar cables como elementos para transmitir el movimiento entre la estructura y el efector final. Esta característica introduce una serie de particularidades desde el punto de vista del modelado y del control del sistema, ya que es necesario garantizar que los cables permanezcan tensionados en todo momento. A pesar de ello, este tipo de robots tiene un peso estructural mucho menor, una mayor escalabilidad y la posibilidad de desplazarse por regiones bastante amplias.

2.3. Configuraciones típicas

Los CDPR pueden adoptar múltiples configuraciones en función del número de cables, puntos de anclaje y grados de libertad por los que se mueve el efector final. En caso de que se quiera mover el efector final en tres dimensiones, el número de cables debe ser superior al número de grados de libertad, con el fin de garantizar que todos los cables utilizados se encuentren tensionados.

En este caso, el efector final se mueve en dos dimensiones, por lo que el número de cables debe ser como mínimo igual al número de grados de libertad, condición que se cumple satisfactoriamente desde el principio.

2.4. Modelo cinemático

El modelo cinemático de un CDPR tiene como objetivo establecer una relación entre la posición del efector final y las longitudes de los cables que lo accionan en cada momento, todo ello haciendo uso de las distancias euclídeas entre cada polea y el efector final.

En este caso, se ha optado por trabajar directamente con la cinemática inversa, ya que lo que se quiere es calcular las longitudes de cada cable siendo conocida la posición del efector final en cada momento. Este enfoque resulta adecuado para su posterior implementación software e integración en ROS 2.

2.5. Control y planificación de trayectorias

El control de un CDPR presenta diversos retos en lo que respecta a los cables y a la continua necesidad de mantenerlos en tensión, ya que en sistemas más avanzados, lo más habitual es incorporar un modelo dinámico, un sistema de control en lazo cerrado y otras estrategias centradas en la tensión de los cables.

Sin embargo, al tratarse de una aplicación mucho más sencilla, en la que el objetivo principal consiste en la validación del comportamiento geométrico del sistema, lo normal es utilizar un modelo cinemático y un sistema de control en lazo abierto, donde la planificación de trayectorias se realice a partir de un conjunto de posiciones objetivo y los cálculos necesarios para pasar por todas ellas de forma continua.

2.6. Frameworks robóticos

ROS 2 se considera actualmente como uno de los frameworks de referencia en el desarrollo de sistemas robóticos, ya que su arquitectura basada en nodos, topics y servicios, hacen mucho más fácil y llevadera la creación de este tipo de sistemas.

Es por eso que la integración de un CDPR en ROS 2 permite separar y distinguir sin ningún problema el modelo cinemático, la lógica de control y la visualización del sistema simulado, la cual se llevará a cabo en RViZ.

2.7. Aplicaciones y líneas de investigación actuales

Los CDPR son utilizados en la actualidad desde sistemas de posicionamiento, manipulación de grandes cargas y plataformas de movimiento, hasta algunos robots de rehabilitación.

Por otro lado, las principales líneas de investigación actuales están centradas en el análisis del espacio de trabajo, la optimización de las tensiones de los cables que componen el sistema, lograr una mayor robustez en el control del sistema y garantizar una interacción segura en todo su entorno.

2.8. Relación con el trabajo de fin de grado desarrollado

Este TFG se sitúa dentro de todo este contexto desde un enfoque simplificado en comparación con aplicaciones ya existentes, haciendo énfasis en el desarrollo software, el modelo cinemático y la integración en ROS 2, lo que permite sentar unas bases sólidas de forma que, en un futuro, se pueda añadir un modelo dinámico, el uso de otro tipo de sensores aparte de los servomotores que actúan como poleas, o la implementación de un sistema de control mucho más avanzado.

Capítulo 3

Plataforma de desarrollo

3.1. Introducción

En este capítulo se presenta una visión global del sistema desarrollado en este TFG, ofreciendo una descripción del CDPR diseñado y de toda la arquitectura que hace posible su funcionamiento, la cual ha permitido validar el modelo cinemático, así como los algoritmos para planificar trayectorias utilizados y su integración en ROS 2. Y aunque gran parte del sistema hardware se encuentre en una fase preliminar, todo su diseño se está teniendo en cuenta desde el principio.

3.2. Funcionamiento de un CDPR bidimensional

El funcionamiento principal de un CDPR bidimensional consiste en el desplazamiento de un efector final, dentro de una región del mismo tipo que el robot, todo ello mediante el accionamiento de dos cables, donde cada uno de ellos conecta el efector final con dos poleas fijas, situadas en cada una de las esquinas superiores de la estructura, lo que permite controlar su posición en función de la longitud de cada uno de los cables.

Desde un punto de vista funcional, el sistema recibe como entrada una o varias posiciones objetivo, a partir de las cuales el sistema va calculando las longitudes de los cables y los ángulos de rotación de cada polea necesarios para hacer que el efector final pase por todas y cada una de las posiciones recibidas.

3.3. Arquitectura

La arquitectura del sistema sigue un enfoque modular y escalable. Esto significa que, aunque toda su implementación abarque un único nodo, posee un diseño que separa las diferentes capas que componen el sistema, las cuales se describen a continuación:

- *Capa de entrada:* Define la posición o conjunto de posiciones (trayectorias) objetivo por las que el efector final debe pasar.
- *Capa cinemática:* Aplica el modelo cinemático para transformar las posiciones recibidas en parámetros físicos del sistema.
- *Capa visual:* Muestra el estado del sistema en cada momento, lo que permite analizar su comportamiento y comprobar que todo está funcionando correctamente.

Toda esta arquitectura facilita la comprensión del sistema y hace que, en caso de que se quiera realizar una ampliación del mismo en un futuro, ésta se pueda llevar a cabo de una forma mucho más sencilla.

Con el objetivo de visualizar el sistema descrito anteriormente, se adjunta a continuación un diagrama que representa la arquitectura general del sistema y la relación entre sus componentes principales.

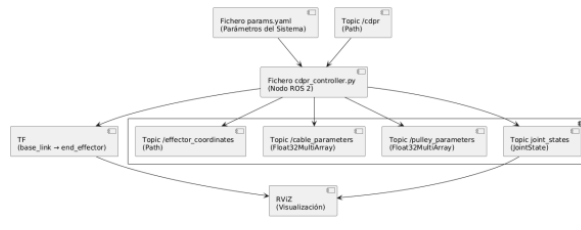


Figura 3.1: Arquitectura general del sistema en ROS 2

Como se puede observar, se tiene una única posición o un conjunto de posiciones objetivo definidas por el usuario, las cuales se envían al nodo principal del sistema implementado, encargado de aplicar el modelo cinemático del robot para calcular las longitudes y los ángulos respecto a la vertical de cada cable, al igual que los ángulos de rotación y la cantidad de cable elongada o recogida por cada polea.

Una vez calculados estos parámetros, toda esta información se publica mediante topics, lo que permite a otros componentes del sistema poder utilizar esta información, mientras se visualiza en RVIZ el estado del sistema en cada momento utilizando el modelo URDF definido, lo que permite observar el comportamiento de todos los elementos que componen el sistema durante la ejecución de una trayectoria.

3.4. Componentes software

El desarrollo software es lo que compone en su gran mayoría al núcleo de este TFG, el cual se ha llevado a cabo en Python, ya que es un lenguaje de programación de alto nivel que se integra perfectamente con ROS 2 y que se caracteriza principalmente por su flexibilidad y legibilidad en código, siendo éstos los factores principales que he tenido en cuenta a la hora de elegir este lenguaje de programación antes que otros, como han sido MATLAB o C++.

El sistema software está compuesto principalmente por los siguientes elementos, los cuales trabajan de forma coordinada con el objetivo de ofrecer una simulación bastante coherente:

- Un nodo ROS 2 encargado de gestionar la ejecución del sistema, el cual recibe como entrada la posición o la trayectoria por la que se quiere desplazar el efector final.
- Un fichero de configuración que permite ajustar los parámetros del sistema y personalizar la visualización.
- Un modelo descriptivo del robot en URDF utilizado para su representación en RViZ.

3.5. Sistema hardware

El sistema hardware comprende una estructura rígida, en este caso una pizarra móvil con ruedas, encargada de sujetar los dos servomotores situados en las esquinas superiores de la misma que actúan como poleas, que a su vez se encargan de accionar los cables que definen la posición del efector final en cada momento, el cual se desplaza libremente dentro de la región bidimensional definida por el área de la pizarra.

Aunque la construcción física del sistema real no esté completa a día de hoy, todo su diseño se ha definido en consonancia con todo el software desarrollado, ya que la posición de cada una de las poleas, el espacio de trabajo equivalente a las dimensiones de la pizarra y el tamaño del efector final se han tenido en cuenta desde las primeras fases de este TFG.

3.6. Flujo general de funcionamiento

En primer lugar, se recibe como entrada del sistema una o varias posiciones objetivo por las que el efector final debe pasar, a partir de las cuales el sistema aplica el modelo cinemático, obteniendo así todos los parámetros necesarios que permiten al sistema accionar los cables con el objetivo de que el efector final pase por todas las posiciones recibidas.

Con todos estos parámetros ya calculados, el sistema se va actualizando en cada momento mientras va publicando información relevante, como la posición del efector final, el valor de la longitud de cada cable y el ángulo girado por cada polea en tiempo real. Todo este flujo de trabajo se repite continuamente hasta que el efector final llegue a la última posición que haya recibido por su entrada.

Es por ello que, para representar el comportamiento del sistema durante su ejecución, se adjunta a continuación un diagrama de flujo que describe todo este proceso.

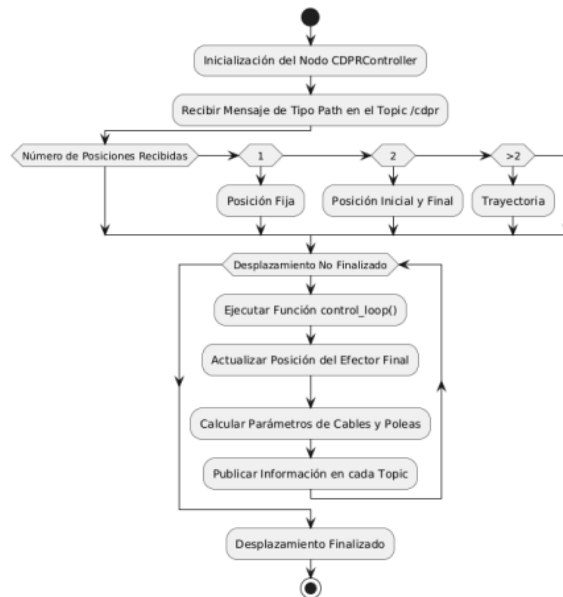


Figura 3.2: Flujo general de funcionamiento del sistema

Como se puede observar, el nodo recibe una única posición o una trayectoria objetivo, a partir de la cual se aplica el modelo cinemático para calcular la longitud de cada cable y los ángulos de rotación de cada polea.

Una vez hecho esto, el sistema publica los resultados obtenidos en topics, permitiendo así conocer el estado del sistema en tiempo real, al igual que en RViZ, donde se puede observar el movimiento del efector final y de las poleas en cada momento de la ejecución del sistema.

3.7. Relación entre módulos

Aunque gran parte de toda la funcionalidad del sistema se implemente en un único nodo, se pueden identificar claramente las relaciones entre todos los módulos que lo componen, como son las posiciones recibidas por su entrada, el cálculo cinemático y su visualización, lo que permite analizar y depurar el comportamiento del sistema de una forma mucho más estructurada, y que además, esta organización se puede seguir teniendo en cuenta en caso de que se quieran hacer ampliaciones en un futuro, como podría ser la adición de otro tipo de sensores en adición a los servomotores que ya se tienen, el paso de tener un sistema de control en lazo abierto a cerrado, o la separación del código en múltiples nodos ROS 2 en lugar de uno solo.

Capítulo 4

Modelo cinemático

4.1. Introducción

El modelo cinemático constituye uno de los pilares fundamentales en el desarrollo del CDPR presentado en este TFG, cuyo objetivo principal es establecer una relación matemática entre la posición (x, y) del efector final, la longitud de cada cable y los ángulos asociados a su orientación y accionamiento mediante poleas.

En este capítulo se presenta el modelo cinemático del sistema, tomando como referencia la implementación del mismo en el nodo ROS 2 desarrollado, cuyo enfoque es puramente geométrico y adecuado para un sistema de dos grados de libertad como el que se tiene y con el que se pueden describir perfectamente todos sus comportamientos.

4.2. Descripción geométrica del sistema

El sistema está formado por un efector final rígido, que se desliza dentro de un plano bidimensional, suspendido mediante dos cables, cada uno de ellos conectado a una polea fija situada en cada una de las esquinas superiores de la estructura, las cuales se encuentran en posiciones ya conocidas dentro del marco de referencia global.

Sabiendo esto, se define un sistema de referencia cartesiano fijo $\{O\}$ asociado a la estructura del robot, donde el espacio de trabajo coincide con el plano XY . Por tanto, la posición del efector final se describe mediante el vector:

$$p = (x, y) \tag{4.1}$$

Ecuación 4.1: Vector de posición del efector final

donde x e y representan las coordenadas del centro geométrico del efector final. Las dimensiones del efector final vienen definidas por un ancho W_e y una altura H_e , donde los puntos de anclaje no coinciden con el centro del efector, sino con sus esquinas superiores.

4.3. Definición de los puntos de anclaje

Siendo $P_i = (X_i, Y_i)$ y $P_d = (X_d, Y_d)$ las posiciones de las poleas izquierda y derecha respectivamente, las cuales son parámetros ya conocidos y fijos en el sistema, se pueden definir los puntos de conexión de cada cable al efector final de la siguiente forma:

$$C_i = (X_{Ci}, Y_{Ci}) = \left(x - \frac{W_e}{2}, y + \frac{H_e}{2} \right) \quad C_d = (X_{Cd}, Y_{Cd}) = \left(x + \frac{W_e}{2}, y + \frac{H_e}{2} \right) \quad (4.2)$$

Ecuación 4.2: Puntos de anclaje entre cada cable y el efector final

Esta definición refleja de forma directa la implementación real del sistema y permite incorporar explícitamente las dimensiones físicas del efector final en el modelo cinemático.

4.4. Cinemática inversa

La cinemática inversa de este sistema consiste en determinar las longitudes de los cables necesarias para situar al efector final en cada una de las posiciones (x, y) por las que se quiere mover el efector final, donde las longitudes de los cables izquierdo y derecho respectivamente, se obtienen como la distancia euclídea entre cada polea y su correspondiente punto de conexión con el efector final de la siguiente forma:

$$L_i = \sqrt{(X_i - X_{Ci})^2 + (Y_i - Y_{Ci})^2} \quad L_d = \sqrt{(X_d - X_{Cd})^2 + (Y_d - Y_{Cd})^2} \quad (4.3)$$

Ecuación 4.3: Longitudes de cada cable

En este modelo los cables se asumen como inextensibles y sin masa, resultando adecuado para el alcance que se le quiere dar a este TFG.

4.5. Cálculo de los ángulos de los cables

Además de la longitud de cada cable, también es importante conocer el ángulo que forma cada cable con la vertical del sistema de referencia, los cuales se calculan de la siguiente forma:

$$\theta_i = \arctan2(Y_i - Y_{Ci}, X_{Ci} - X_i) \quad \theta_d = \arctan2(Y_d - Y_{Cd}, X_{Cd} - X_d) \quad (4.4)$$

Ecuación 4.4: Orientación angular de cada cable

En términos de software, los ángulos obtenidos en radianes se pasan a grados, de forma que puedan adaptarse lo mejor posible a la hora de visualizar el sistema en movimiento desde RViZ.

4.6. Relación cables-poleas

El accionamiento de los cables se realiza mediante el uso de dos poleas de radio R_p , lo que implica que un cambio en la longitud de uno de los cables se traduce directamente en una rotación de la polea correspondiente.

Siendo ΔL la diferencia de longitud del cable entre dos posiciones consecutivas del efector final, el ángulo de rotación de la polea se puede obtener como:

$$\Delta\theta = \frac{\Delta L}{R_p} \quad (4.5)$$

Ecuación 4.5: Relación entre desplazamiento y rotación de cada cable y polea

Por tanto, este modelo permite relacionar el espacio cartesiano del efector final con el espacio articular del sistema, definido por los ángulos de rotación de cada una de las poleas.

4.7. Consideraciones y limitaciones

El modelo cinemático presentado puede considerarse como una versión simplificada, ya que siempre se tiene en cuenta que los cables nunca se verán afectados por ningún tipo de fuerza externa o porque éstos no poseen ningún tipo de propiedad elástica, además de la ausencia de fricción en las poleas.

Sin embargo, a pesar de estas simplificaciones, el modelo cinemático descrito recoge requisitos más que suficientes para describir el comportamiento general del sistema y demostrar que es válido tanto para realizar pruebas simuladas como reales, las cuales también deben tenerse en cuenta a la hora de plantear mejoras en un futuro.

Capítulo 5

Simulación y validación del modelo

5.1. Introducción

Una vez desarrollado el modelo cinemático del CDPR, se ha llevado a cabo un proceso de validación y simulación con el objetivo de comprobar las expresiones matemáticas obtenidas del modelo cinemático en una primera implementación software. Esto permite verificar el comportamiento del sistema para así poder detectar posibles errores de planteamiento y evitar que se vayan acarreando a la hora de integrar el sistema completo en ROS 2.

En el caso de este TFG, las pruebas del modelo cinemático se han ido realizando de forma progresiva mediante su implementación en tres lenguajes de programación diferentes: MATLAB, Python y C++.

5.2. Simulación inicial en MATLAB

MATLAB es el primer lenguaje de programación que se ha utilizado para validar el modelo cinemático, ya que éste se considera idóneo para tratar expresiones matemáticas y obtener una rápida visualización de los resultados. Es por ello que, en esta fase inicial, la simulación se ha centrado exclusivamente en verificar que el efector final se ubica correctamente en una posición (x, y) específica.

Esta primera implementación permite introducir directamente la posición (x, y) en la que se quiere mostrar el efector final, además de ofrecer una representación gráfica de su posición exacta dentro del espacio de trabajo junto con toda la estructura del sistema y la disposición de los cables.

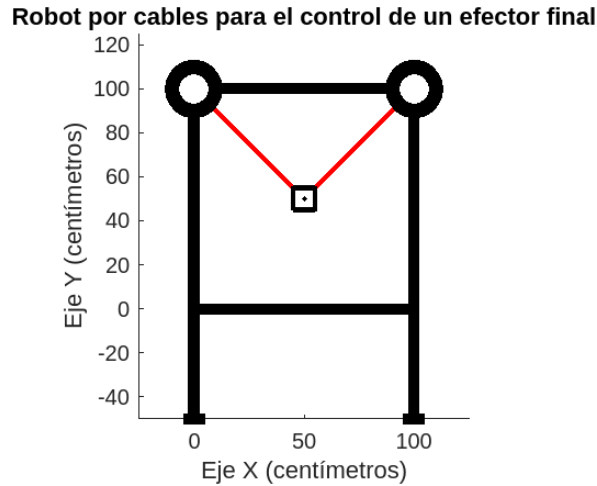


Figura 5.1: Simulación inicial en MATLAB

Con esto se ha cumplido el objetivo de esta primera versión, que no comprende la simulación del movimiento del efector final ni la planificación de trayectorias, sino que lo que se busca es verificar que el modelo cinemático sitúe al efector final en la posición deseada y que las longitudes de los cables calculadas sean coherentes con la geometría del sistema, sirviendo así como validación conceptual del modelo y como punto de partida para implementaciones posteriores.

5.3. Implementación del modelo en Python

Una vez validado el primer modelo en MATLAB, se ha desarrollado una implementación más avanzada en Python, la cual ya incluye animaciones y la posibilidad de definir múltiples posiciones objetivo para así generar trayectorias continuas que conecten dichos puntos.

Esta segunda implementación permite introducir una secuencia de posiciones objetivo para el efector final dentro del espacio de trabajo. A partir de estas posiciones, se pueden interpolar trayectorias rectilíneas entre puntos consecutivos, mientras se calculan en cada instante las longitudes y los ángulos de cada cable utilizando las ecuaciones correspondientes de cinemática inversa.

Además, esta implementación incluye una representación gráfica del sistema, en la que se muestra la estructura, la trayectoria realizada, y la variación del movimiento del efector final y los cables en todo momento, facilitando así la comprensión del funcionamiento del sistema y permitiendo observar de forma intuitiva cómo va evolucionando el movimiento del efector final y de los cables. Es por ello que esta segunda versión ya se aproxima mucho más al funcionamiento que se implementará posteriormente en ROS 2.

Para una mejor comprensión del comportamiento dinámico del sistema mostrado en la figura, se adjunta un enlace de descarga a un vídeo demostrativo al que se puede acceder haciendo clic aquí.

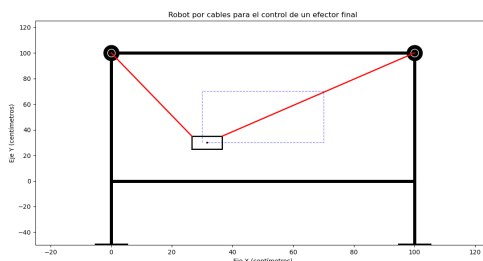


Figura 5.2: Implementación y simulación en Python

5.4. Implementación del modelo en C++

En paralelo a la implementación en Python, se ha desarrollado una versión del modelo cinemático en C++. Esta implementación reproduce la misma lógica que la vista en Python, permitiendo la definición de varias posiciones objetivo y el cálculo de la longitud de los cables en cada momento para recorrer una trayectoria.

Esta simulación se ha llevado a cabo únicamente con fines de verificación y comparación para asegurar que el modelo cinemático es independiente del lenguaje de programación utilizado, y que todas las ecuaciones que lo forman sean consistentes. Y aunque la visualización gráfica en C++ no sea tan completa como la de Python, también es útil para validar todos los cálculos numéricos realizados.

Para una mejor comprensión del comportamiento dinámico del sistema mostrado en la figura, se adjunta un enlace de descarga a un vídeo demostrativo al que se puede acceder haciendo clic aquí.

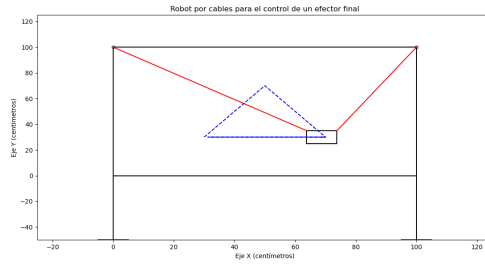


Figura 5.3: Implementación y simulación en C++

5.5. Comparación y validación de resultados

Para validar el modelo cinemático, se han comparado los resultados obtenidos en las tres implementaciones, y todas ellas presentan una muy buena concordancia, lo que deja más que confirmado que el modelo cinemático utilizado es totalmente correcto.

Sin embargo, existen algunas diferencias en cuanto a funcionalidad y flexibilidad. MATLAB se ha utilizado como herramienta para llevar a cabo una primera validación, limitándose únicamente al posicionamiento de una única posición objetivo, mientras que las implementaciones realizadas en Python y en C++ permiten la generación de trayectorias definidas por múltiples posiciones objetivo.

Y es por eso que, tras este análisis comparativo, he decidido utilizar Python como lenguaje principal a la hora de implementar el sistema en ROS 2, debido a la facilidad que supone desarrollar en Python ofreciendo la posibilidad de poder integrarlo directamente en ROS 2, manteniendo el mismo comportamiento cinemático que en C++.

5.6. Análisis de errores y limitaciones

Durante el proceso de simulación, se han identificado algunas limitaciones que no afectan al sistema debido al enfoque que se le ha dado al mismo, ya que al tratarse de un modelo puramente cinemático, no se consideran efectos dinámicos como la masa del efector final, la elasticidad de los cables o la fricción de las poleas.

Por otro lado, todas las simulaciones se realizan en lazo abierto, lo que implica que todos los resultados obtenidos dependen directamente de la precisión del modelo y de los parámetros geométricos definidos. Todas estas limitaciones no invalidan ningún aspecto del sistema ya definido, pero es importante tenerlas en cuenta a la hora de interpretar los resultados cuando se esté implementando en un entorno físico real.

Capítulo 6

Integración del sistema en ROS 2

6.1. Introducción

Una vez validado el modelo cinemático del CDPR en todas las simulaciones mencionadas en el capítulo anterior, el siguiente paso consiste en la integración completa del sistema en ROS 2 (Robot Operating System 2), lo que va a permitir dotar al sistema de una arquitectura modular y escalable, facilitando así su visualización a la hora de probarlo en un sistema físico real.

En este capítulo se describe la estructura general del paquete ROS 2 desarrollado, el funcionamiento del nodo que implementa toda la parte de control del CDPR, todo lo relacionado con las comunicaciones realizadas mediante topics y la visualización del sistema en RViZ.

Toda esta integración se ha llevado a cabo haciendo uso de la distribución Jazzy de ROS 2 y de Python como lenguaje de programación, tal y como se mencionó en el capítulo anterior.

6.2. Estructura del paquete ROS 2

El sistema se ha organizado en un paquete ROS 2 llamado *cdpr_2d*, cuya organización y estructura siguen un modelo estándar, facilitando así la comprensión del código y lo que permite en un futuro modificar o ampliar componentes concretos del sistema sin tener que realizar grandes cambios.

En este paquete ROS 2 se pueden distinguir como elementos principales:

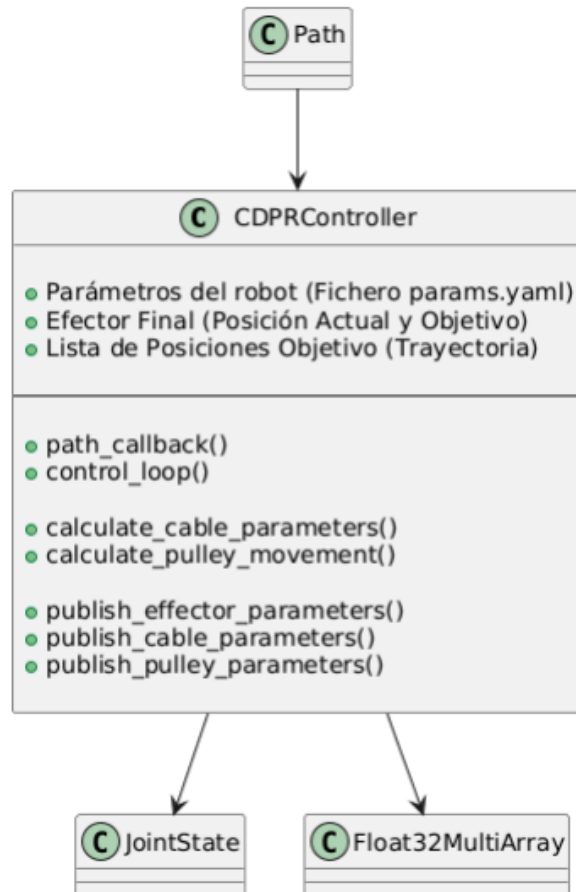
- Fichero *cdpr_controller.py*: Nodo principal que implementa el funcionamiento completo del sistema que incluye la planificación de trayectorias, todos los cálculos cinemáticos y la publicación de resultados haciendo uso de diferentes topics.
- Directorio *config/*: Contiene un fichero de configuración llamado *params.yaml* en el que se definen los parámetros del sistema, y otro fichero llamado *robot.rviz* encargado de configurar la visualización del sistema en RViZ.
- Directorio *description/*: Contiene los modelos del sistema en URDF en el fichero *robot.urdf*, y en XACRO en el fichero *robot.xacro*.
- Directorio *launch/*: Contiene un fichero de lanzamiento llamado *robot_state_publisher.launch.py*, que permite inicializar de forma conjunta tanto la simulación como la visualización del sistema en RViZ.
- Directorio *test/*: Contiene varios ficheros de test automáticos para verificar que el estilo seguido en el código es limpio y correcto.

6.3. Nodo de control del CDPR

El sistema integrado en ROS 2 tiene como núcleo el nodo *cdpr_controller*, encargado de recibir las posiciones objetivo por las que se quiere desplazar el efector final para así poder aplicarlas en el modelo cinemático desarrollado, con el objetivo de obtener todos los parámetros relacionados con los cables y las poleas del sistema.

El nodo se inicializa y se mantiene activo en todo momento para que pueda recibir todos los mensajes que contienen cada una de las posiciones objetivo, cuyo diseño y funcionalidad se concentran en un único nodo con el objetivo de simplificar la integración y el análisis del sistema completo.

Por ello, se adjunta a continuación un diagrama de clases simplificado que describe los elementos más importantes del nodo *cdpr_controller*, con el objetivo de facilitar la comprensión del mismo, ya que éste se considera el núcleo del sistema.

Figura 6.1: Estructura del nodo `cdpr_controller`

Como se puede observar, la clase principal corresponde al nodo *cdpr_controller*, encargado de gestionar la ejecución del sistema dentro de ROS 2.

Este nodo recibe todas las posiciones o trayectorias objetivo por las que se quiere desplazar el efector final a través de un topic y haciendo uso del modelo cinemático para calcular todos los parámetros asociados a los cables y a las poleas, para posteriormente, publicar toda esta información en distintos topics.

Es por ello que esta organización es la que permite concentrar toda la lógica del sistema en un único nodo siguiendo una estructura que facilita la comprensión del mismo, y donde podrían llevarse a cabo ampliaciones de cara al futuro.

Para una mejor comprensión del comportamiento dinámico del sistema mostrado en la figura, se adjunta un enlace de descarga a un vídeo demostrativo al que se puede acceder haciendo clic aquí.

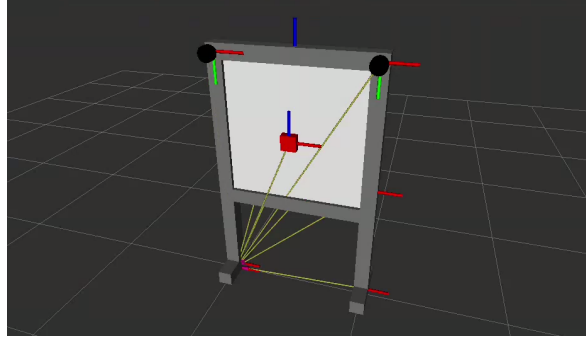


Figura 6.2: Visualización en RViZ durante la ejecución de una trayectoria

6.4. Comunicación mediante topics

La comunicación entre el nodo *cdpr_controller* y el resto del sistema se lleva a cabo utilizando topics que siguen un modelo publicador-suscriptor, lo que permite diferenciar los distintos componentes del sistema y monitorear el estado del mismo en tiempo real.

Por ello, se adjunta a continuación un diagrama de secuencia que muestra el intercambio de mensajes y cómo se produce esta comunicación entre los distintos componentes del sistema.

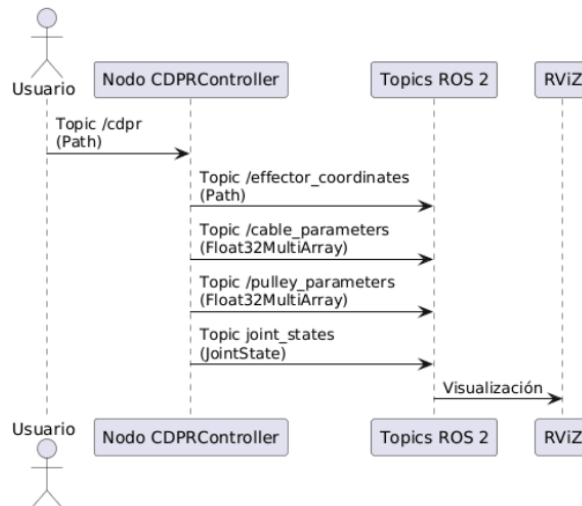


Figura 6.3: Comunicación mediante topics en ROS 2

En primer lugar, el usuario publica una única posición o un conjunto de posiciones objetivo en el topic correspondiente. Después, el nodo *cdpr_controller* recibe este mensaje y va procesando cada una de las posiciones objetivo que ha recibido haciendo uso del modelo cinemático. Y por último, el nodo *cdpr_controller* publica en cada topic correspondiente mensajes sobre la posición del efector final o parámetros asociados a los cables y a las poleas, los cuales son también utilizados por RViZ para poder representar el estado del sistema de forma visual y en tiempo real.

El nodo *cdpr_controller* se suscribe al topic */cdpr*, que recibe mensajes del tipo *nav_msgs/msg/Path*, que contiene una única o una secuencia de posiciones objetivo por las que se quiere desplazar el efector final. Una vez recibida esta información, el nodo publica en distintos topics:

- Topic */effector_coordinates*: Muestra la posición en cada momento del efector final en el plano XY.
- Topic */cable_parameters*: Muestra la longitud y el ángulo que forman con la vertical de cada uno de los cables.
- Topic */pulley_parameters*: Muestra la diferencia de longitudes de cada cable y la diferencia de ángulos de rotación de las poleas entre dos posiciones consecutivas.

Estos topics permiten observar de forma detallada el comportamiento interno del sistema en todo momento

6.5. Gestión de trayectorias y movimientos

Este sistema tiene implementados diferentes modos de comportamiento en función del número de puntos recibidos en el mensaje de tipo *nav_msgs/msg/Path*.

Cuando se recibe una única posición, el sistema interpreta la posición recibida como un posicionamiento fijo del efector final. En caso de que haya recibido dos posiciones, lo que se genera es un desplazamiento entre una posición inicial y una posición final. Y por último, si recibe tres o más posiciones, el sistema lo interpreta como una trayectoria compuesta por todas las posiciones que se le han pasado y por las que el efector final debe pasar en el orden especificado.

Para este último caso, el nodo interpola posiciones intermedias y aplica el modelo cinemático para calcular los parámetros correspondientes en todo momento para cada segmento de la trayectoria. Esto permite simular un movimiento continuo y coherente del efector final dentro del espacio de trabajo.

6.6. Visualización del sistema en RViZ

La integración del sistema y su visualización en RViZ son claves en el desarrollo del mismo, ya que permiten observar de forma mucho más clara el comportamiento del sistema. Para ello, se ha definido un modelo URDF que describe la estructura fija, los cables, las poleas y el efector final.

El lanzamiento conjunto del nodo *cdpr_controller* y del *robot_state_publisher* permiten actualizar en todo momento la visualización del robot en RViZ según se va ejecutando cada trayectoria, cuya configuración facilita la observación de los distintos elementos que componen el sistema y su evolución cuando se encuentran en movimiento.

6.7. Compilación y ejecución del sistema

Para llevar a cabo la ejecución del sistema, se ha definido un procedimiento de instalación basado en colcon, donde el repositorio de Github oficial de este TFG actúa como el directorio *src/* de un workspace, lo que permite compilar y ejecutar el paquete directamente.

Una vez compilado el workspace, la ejecución del sistema completo consiste en la ejecución del nodo *cdpr_controller* y el lanzamiento del entorno de visualización en RViZ. Por otro lado, se publican en el topic correspondiente los mensajes de tipo *nav_msgs/msg/Path* que contienen las posiciones objetivo por las que se desea desplazar el efector final, lo que facilita las pruebas y la validación del comportamiento del sistema.

Este procedimiento asegura la reproducción correcta de todas las pruebas que se lleven a cabo y simplifica la puesta en marcha del sistema en todos los entornos utilizados.

Capítulo 7

Desarrollo hardware

7.1. Introducción

...

7.2. Diseño mecánico del sistema

...

7.3. Sistema electrónico

...

7.4. Programación del microcontrolador

...

7.5. Integración hardware-software

...

7.6. Consideraciones prácticas y limitaciones del prototipo

...

7.7. Resumen

...

Capítulo 8

Pruebas experimentales y validación

8.1. Introducción

...

8.2. Metodología de pruebas

...

8.3. Pruebas de posicionamiento del efector final

...

8.4. Prueba de trayectorias

...

8.5. Comparación entre simulación y resultados

...

8.6. Análisis de errores y fuentes de incertidumbre

...

8.7. Resumen

...

Capítulo 9

Discusión de resultados

9.1. Introducción

...

9.2. Evaluación del cumplimiento de los objetivos

...

9.3. Análisis de los resultados de la simulación

...

9.4. Análisis de los resultados experimentales

...

9.5. Comparación entre comportamiento teórico y real

...

9.6. Limitaciones del sistema desarrollado

...

9.7. Propuesta de mejora y líneas de evolución

...

9.8. Resumen

...

Capítulo 10

Conclusiones y mejoras a futuro

10.1. Introducción

...

10.2. Conclusiones generales

...

10.3. Grado de cumplimiento de los objetivos

...

10.4. Aportaciones del trabajo

...

10.5. Limitaciones finales del sistema

...

10.6. Mejoras a futuro

...

10.7. Cierre del trabajo

...

Capítulo 11

Uso e impacto de la IA y los LLM en el TFG

11.1. Introducción

En los últimos años, se han desarrollado numerosas herramientas de Inteligencia Artificial, concretamente aquellas relacionadas con los Modelos de Lenguaje de Gran Escala, también conocidos como Large Language Models (LLMs), lo que ha supuesto un cambio significativo en la manera de abordar una amplia diversidad de tareas, como por ejemplo, tareas de planificación, redacción y programación, en el caso de este TFG.

En el ámbito académico, estas herramientas se han convertido en un recurso habitual para los estudiantes, permitiendo mejorar la organización del trabajo o resolver dudas concretas. Sin embargo, el uso de estas herramientas se debe realizar de forma responsable, con el objetivo de garantizar que todo el contenido generado y proporcionado por las mismas sea comprendido, validado y adaptado dentro del trabajo o proyecto que se esté desarrollando.

En este capítulo se describen todos los usos que se han hecho de este tipo de herramientas, en qué medida, y en qué fases específicas han tenido mayor impacto dentro de este TFG.

11.2. Herramientas utilizadas

La única herramienta utilizada en este TFG basada en Inteligencia Artificial ha sido ChatGPT, un modelo de lenguaje diseñado para ayudar a resolver y comprender tareas mediante la generación de texto.

En el caso de este TFG, el uso de esta herramienta se ha limitado a la organización del trabajo, la estructuración de contenidos y la resolución de pequeñas dudas puntuales relacionadas con el desarrollo software. En ningún caso se ha utilizado como sustituto del trabajo técnico propio, sino como un recurso complementario que ha facilitado abordar fases del proyecto relacionadas con los casos anteriormente mencionados.

11.3. Ámbitos de utilización

Uno de los principales usos de *ChatGPT* en este TFG ha sido como herramienta de apoyo para organizar y estructurar el código fuente desarrollado. Concretamente, para:

- Proponer formas de organizar el código de forma mucho más legible.
- Resolver dudas puntuales relacionadas con la implementación de ciertas funciones.
- Sugerir buenas prácticas de programación en Python para su posterior integración en ROS 2.

Este primer uso ha permitido obtener una estructuración del código clara y coherente, lo que ha facilitado su comprensión.

Otro de los usos de *ChatGPT* en este TFG ha sido para definir la estructura de la memoria del mismo, además de servir como apoyo para:

- Definir un índice base para la memoria.
- Organizar los capítulos de forma que los contenidos vistos en cada uno de ellos sean progresivos respecto a los anteriores.
- Ajustar el tamaño de cada capítulo en función de la cantidad de trabajo llevada a cabo.

Este segundo uso ha permitido obtener una estructura alineada con el desarrollo de este TFG en lo que respecta a la memoria.

Y el último uso de *ChatGPT* en este TFG ha sido relacionado con la planificación temporal de las distintas tareas llevadas a cabo, lo que ha permitido:

- Estimar un tiempo límite aproximado para abordar cada tarea.
- Organizar las tareas de forma secuencial y con una cantidad de trabajo incremental.
- Ajustar la planificación en función del progreso desarrollado en todo momento.

Este tercer y último uso ha resultado útil para asegurar el cumplimiento de los plazos establecidos para todas las tareas desarrolladas en este TFG.

11.4. Nivel de dependencia

El uso de *ChatGPT* en este TFG ha sido limitado y controlado en todo momento, de forma que todas sus funciones han sido únicamente auxiliares dentro del proceso de desarrollo.

Todo lo que respecta a decisiones técnicas, el diseño del sistema, la formulación del modelo cinemático, y la implementación y validación del mismo, han sido realizadas en su totalidad por el autor de este TFG. *ChatGPT* se ha utilizado únicamente como una herramienta de apoyo para mejorar la organización del mismo y resolver dudas puntuales.

Por tanto, se puede afirmar que en ningún momento ha existido una dependencia total o directa de la herramienta, cuyo uso se ha limitado a mejorar la eficiencia del desarrollo de este TFG.

11.5. Impacto en el desarrollo del TFG

El impacto del uso de herramientas de Inteligencia Artificial en este TFG ha sido positivo, principalmente en aspectos relacionados con la organización y la planificación del mismo. Es por ello que hay que destacar tres grandes beneficios obtenidos:

- Mejora de la estructuración del trabajo tanto a nivel de código como a nivel de documentación.
- Optimización del tiempo al facilitar la resolución de dudas concretas y la planificación de tareas.
- Mayor claridad en la organización del contenido.

Sin embargo, este impacto se ha limitado única y exclusivamente a aspectos de apoyo, sin influir en ningún momento en el núcleo de este TFG.

11.6. Limitaciones y consideraciones éticas

El uso de cualquier herramienta basada en Inteligencia Artificial conlleva una serie de limitaciones que se deben tener en cuenta durante su uso.

En primer lugar, hay que tener en cuenta que este tipo de herramientas puede generar respuestas incorrectas o incompletas. Es por ello que toda la información proporcionada por ChatGPT ha sido revisada y validada antes de ser utilizada o implementada en este TFG.

En segundo lugar, es importante mencionar que todos los usos de esta herramienta se han realizado de forma responsable, evitando en todo momento la generación de contenido sin haber obtenido previamente una comprensión del mismo, lo que ha resultado clave para garantizar la calidad y originalidad de este TFG.

Y por último, destacar que todos los usos de esta herramienta se han realizado respetando principios académicos y de autoría en todo momento, lo que ha permitido reflejar todo el esfuerzo dedicado y todos los conocimientos adquiridos a lo largo del grado.

11.7. Conclusión

El uso de *ChatGPT* en este TFG se ha traducido en un apoyo únicamente complementario durante el desarrollo del mismo, centrándose en tareas de organización, estructuración y planificación, lo que ha permitido mejorar la calidad del proyecto y optimizar el tiempo de desarrollo de cada tarea, todo ello sin sustituir en ningún momento cualquier trabajo técnico propio.

Por ello, se puede concluir que el uso responsable de este tipo de herramientas puede aportar mucho más valor a todo el proceso de desarrollo en un proyecto académico, siempre que se mantenga un control riguroso sobre su uso y se garantice la comprensión, revisión y validación de toda la información proporcionada.

Capítulo 12

Conceptos comunes abordados en grado y TFG

12.1. Introducción

A lo largo del Grado en Ingeniería de Robótica Software, se ha adquirido una amplia variedad de conocimientos en múltiples disciplinas, desde fundamentos matemáticos y físicos hasta áreas como la programación, la robótica, los Sistemas Distribuidos o la Inteligencia Artificial.

Es por ello que este TFG no se encuentra aislado de lo visto en el grado, ya que integra gran parte de todos estos conocimientos, aplicándolos de forma conjunta para hacer posible el diseño, implementación y validación de un sistema robótico real. Dicho de esta manera, se puede entender que este TFG actúa como punto de unión con gran parte de los conceptos aprendidos durante toda la formación académica.

En este capítulo se explican todas las relaciones entre las asignaturas del Grado en Ingeniería de Robótica Software y los distintos componentes de este TFG, destacando cómo y en qué medida ha influido cada una de ellas en el resultado final.

12.2. Relación con asignaturas de primer curso

Las asignaturas de primer curso han proporcionado los fundamentos necesarios sobre los que se apoya todo el desarrollo posterior de este TFG.

Las asignaturas de *Álgebra*, *Cálculo* y *Fundamentos Físicos de la Robótica* han resultado esenciales para poder formular el modelo cinemático del sistema. El uso de vectores, distancias euclídeas, relaciones geométricas, magnitudes físicas de posición y movimiento, y relaciones espaciales, han permitido describir la posición del efector final en el espacio y establecer relaciones entre todas las variables físicas que componen el sistema.

Y por otro lado, asignaturas como *Fundamentos de la Programación* y *Algoritmos y Estructuras de Datos* han sido claves para llevar a cabo el desarrollo software del sistema, donde la capacidad de estructurar código, definir funciones y diseñar algoritmos, ha resultado útil para implementar el modelo cinemático y la planificación de trayectorias.

12.3. Relación con asignaturas de segundo curso

Las asignaturas de segundo curso introducen conceptos orientados al diseño de sistemas y al ámbito específico de la robótica, las cuales establecen una relación directa en el desarrollo de este TFG.

La asignatura de *Diseño Software* ha influido directamente en la organización modular del sistema desarrollado, donde la estructuración del código y la claridad en la arquitectura del sistema reflejan los conceptos vistos en esta asignatura.

Por otro lado, la asignatura de *Sensores y Actuadores* se relaciona directamente con el desarrollo del sistema hardware, concretamente en el uso de los servomotores encargados de accionar las poleas, permitiendo comprender cómo se traduce una señal de control en un movimiento físico real.

Y por último, las asignaturas de *Arquitecturas Software para Robots* y *Sistemas Operativos* se relacionan directamente con la integración del sistema en ROS 2, ya que introducen conceptos como la comunicación entre componentes, el diseño de sistemas robóticos complejos, la gestión de recursos y la ejecución concurrente.

12.4. Relación con asignaturas de tercer curso

Las asignaturas de tercer curso son las que constituyen el núcleo de este TFG, especialmente en lo que respecta a robótica y sistemas software avanzados.

Las asignaturas de *Robótica Móvil*, *Modelado y Simulación de Robots*, y *Sistemas Empotrados y de Tiempo Real* tienen una relación directa con el desarrollo del modelo cinemático y su validación, permitiendo comprender cómo modelar el movimiento de un robot y cómo simular su comportamiento para posteriormente integrarlo en un sistema real, donde aspectos como la ejecución en tiempo real y la interacción con dispositivos físicos cobran una gran importancia.

Por otro lado, la asignatura de *Sistemas Distribuidos y Concurrentes* refleja claramente el uso de ROS 2, concretamente en la organización del sistema en nodos que se comunican entre sí a través de topics, todo ello mientras se sigue un modelo distribuido.

Y por último, la asignatura de *Planificación y Sistemas Cognitivos* posee una relación directa con la generación de trayectorias, ya que permite al sistema generar movimientos continuos a partir de múltiples posiciones objetivo.

12.5. Relación con asignaturas de cuarto curso

Las asignaturas de cuarto curso aportan una visión más aplicada y especializada, aunque también se pueden ver reflejadas en el desarrollo de este TFG.

La asignatura de *Robótica de Servicio* permite contextualizar el sistema desarrollado en aplicaciones reales, por ejemplo, en sistemas de posicionamiento o de manipulación ligera, donde los CDPR pueden tener un papel relevante.

Y por otro lado, la asignatura de *Mecatrónica* resulta clave en la integración entre hardware y software, uno de los aspectos fundamentales de este TFG, ya que permite plantear un sistema físico controlado a partir del sistema software desarrollado.

12.6. Integración global de conocimientos en el TFG

El desarrollo de este TFG es un ejemplo claro de la necesidad de combinar elementos de diferentes áreas para construir un sistema completo y funcional.

Por ello, se han aplicado conceptos matemáticos para definir el modelo cinemático, conocimientos de programación para implementar el modelo diseñado en distintos lenguajes y entornos, conceptos de arquitectura software para integrar el sistema en ROS 2, y conceptos de hardware para implementar el sistema en físico.

Es por todo esto que toda esta integración de conocimientos permite abordar el problema de forma global y comprender cómo interactúan entre sí todos los componentes que forman el sistema completo.

12.7. Conclusión

Este TFG representa una combinación de gran parte de los conocimientos adquiridos a lo largo del Grado en Ingeniería de Robótica Software.

A través del desarrollo de un sistema basado en un CDPR, se han podido poner en práctica conceptos de múltiples asignaturas, lo que ha permitido demostrar la capacidad de integrar conocimientos teóricos y aplicarlos en un contexto práctico y real.

Es por ello que este proceso no sólo ha permitido consolidar los aprendizajes adquiridos a lo largo del grado, también han permitido desarrollar una visión más completa del ámbito de la robótica, la cual requiere la combinación de diferentes áreas de conocimiento para la resolución de problemas complejos.

Bibliografía

- [1] IEEE. Cable-driven parallel robot related research. 2019.
- [2] YouTube. Modelado cinemático - recurso audiovisual 1. <https://www.youtube.com/watch?v=Lgs8SA1bLrc>.
- [3] YouTube. Modelado cinemático - recurso audiovisual 2. <https://www.youtube.com/watch?v=PB4f1p8K2uk>.
- [4] MathWorks. *MATLAB Documentation*.
- [5] Python Software Foundation. *Python 3 Documentation*.
- [6] DevDocs. *C++ Reference Documentation*.
- [7] Open Robotics. *ROS 2 Jazzy Documentation*.
- [8] Open Robotics. *URDF Tutorials*.
- [9] Open Robotics. *RViz Tutorials*.
- [10] Colcon Project. *Colcon Documentation*.
- [11] GitHub. *GitHub Documentation*.
- [12] Open Robotics. *ROS 2 nav_msgs/Path Message Documentation*.
- [13] micro-ros esp-idf component. https://github.com/micro-ROS/micro_ros_esp_idf_component/tree/jazzy.
- [14] eProsima. *Micro XRCE-DDS Documentation*.
- [15] Alberto León. Cable driven parallel robot 2d. https://github.com/aleon2020/cable_driven_parallel_robot_2d.